DTI201 • สถาปัตยกรรมระบบสำหรับ Full-Stack

สถาปัตยกรรม Full-Stack และ Container



จาก React และ Book API ถึงระบบปฏิบัติการและฐานข้อมูล

Container → Docker → Docker Compose

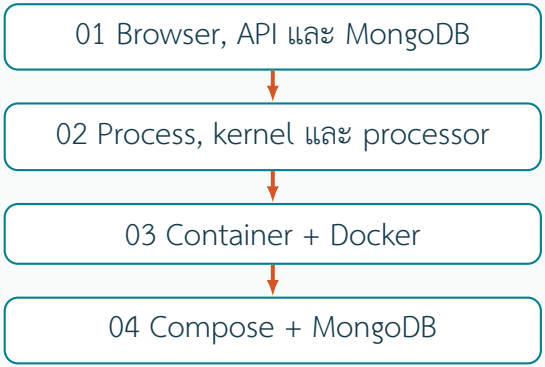
เมื่อจบคาบ เราจะอธิบายการรัน Book API ได้

อธิบายหน้าที่ของ browser, frontend, API และ database จากคำขอหนังสือหนึ่งครั้ง

เชื่อม Book API และ Node.js กับ process, system call, kernel, memory และ processor

เปรียบเทียบ **image กับ container**, ทดลองคำสั่ง Docker แล้วอ่านแผนบริการของ Docker Compose

สุดท้ายอธิบาย port และที่เก็บข้อมูลที่ MongoDB ใช้ใน Lab 10 ด้วยหลักฐานจากใบงาน



ลองคิดหรือทดลอง: เมื่อจบคาบ เราควรชี้ให้ได้ว่า MongoDB เก็บข้อมูลที่ใด

การรันแอปต้องเตรียม runtime และ dependencies

Book API ต้องมีโค้ด, Node.js และแพ็กเกจที่ระบุไว้ใน `package.json` เครื่องที่มีเพียงโค้ดอาจเปิดแอปไม่ได้

อุปกรณ์ของร้านกาแฟเป็นภาพเปรียบเทียบ: ไม่ว่าจะเปิดในบ้าน เต็นท์ หรือรถบ้าน ร้านต้องเตรียมเครื่องชงและแหล่งไฟให้พร้อม

Image ช่วยบรรจุซอฟต์แวร์บางส่วนให้ทำซ้ำได้; configuration, ข้อมูล, เครือข่าย และระบบที่รันยังต้องจัดการ



ลองคิดหรือทดลอง: เมื่อย้าย Book API ไปเครื่องใหม่ ต้องตรวจอะไรนอกจากไฟล์โค้ด?

Image บรรจุแอปและ dependencies เป็นชุด

รถบ้านร้านกาแฟเตรียมเคาน์เตอร์ เครื่องชง และอุปกรณ์ไว้ด้วยกัน จึงเคลื่อนย้ายชุดอุปกรณ์ไปเปิดร้านในที่ใหม่ได้สะดวกขึ้น

แต่รถยังต้องอาศัยถนน จุดจอด ไฟ และน้ำที่ใช้ร่วมกับสถานที่. การเตรียมรถไม่ทำให้ทุกสถานที่พร้อมโดยอัตโนมัติ

ใน Docker, image บรรจุไฟล์แอป runtime และ dependencies; container ที่สร้างจาก image ใช้ kernel ของระบบที่รัน



ลองคิดหรือทดลอง: อะไรอยู่ในรถ และอะไรยังต้องเตรียมจากสถานที่ใหม่?

ช่วงที่ 1 / 4

โปรแกรมต้องอาศัยหลายชั้นจึงทำงานได้

ไล่จาก Book API ลงไปถึงตัวประมวลผล

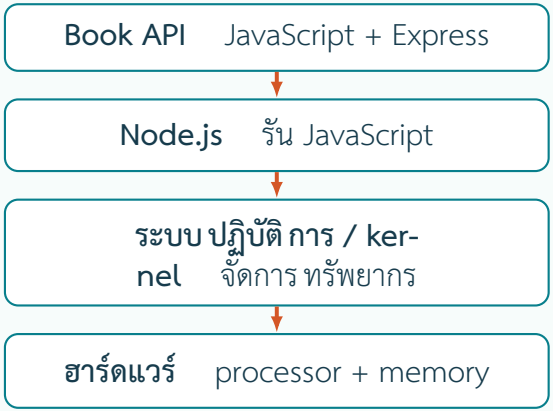
Browser → Express → MongoDB → Storage

Book API ต้องใช้ runtime และระบบปฏิบัติการ

แอป Book API เป็นโค้ด JavaScript ที่เราสร้างเพื่อรับคำขอและตอบข้อมูลหนังสือ

Node.js อ่านและรัน JavaScript; Express เป็นแพ็กเกจที่ช่วยเขียนเว็บเซิร์ฟเวอร์. แอปขอใช้ไฟล์ เครือข่าย และหน่วยความจำผ่านระบบปฏิบัติการ

ด้านล่างสุด CPU หรือ processor ทำงานตามคำสั่งของเครื่อง โดยใช้หน่วยความจำและอุปกรณ์ร่วมด้วย



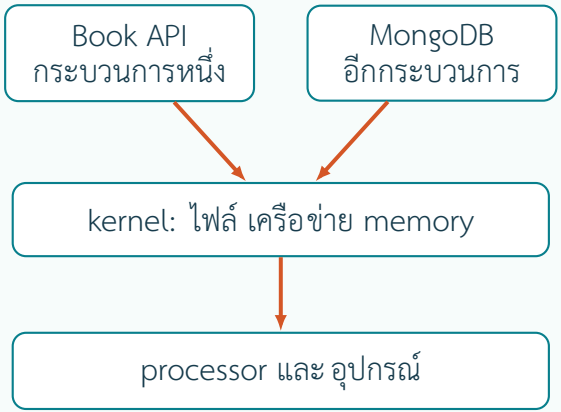
ลองคิดหรือทดลอง: ถ้าไม่มี Node.js เครื่องจะรัน Book API JavaScript ของเราได้อย่างไร?

Kernel จัดการทรัพยากรให้โปรแกรมใช้

Kernel เป็นแกนของระบบปฏิบัติการ. โปรแกรมขอให้ kernel ช่วยจัดการงาน เช่น เปิดไฟล์ รับส่งข้อมูลเครือข่าย และใช้หน่วยความจำ

โปรแกรมหลายตัวใช้ kernel เดียวกันได้ โดยระบบจัดการสิ่งที่แต่ละกระบวนการมองเห็นและใช้ได้

ชั้นนี้สำคัญเพราะ Linux containers ที่รันบนระบบเดียวกันใช้ Linux kernel ร่วมกัน



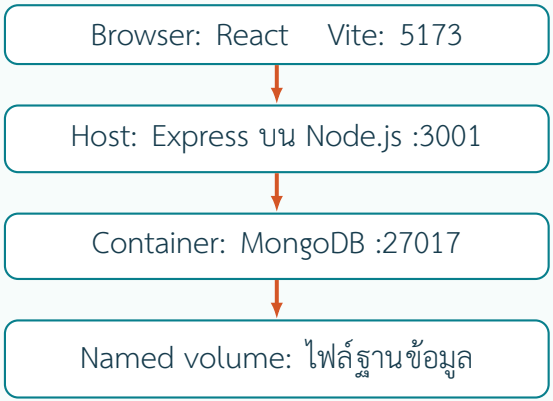
ลองคิดหรือทดลอง: Book API กับ MongoDB ต้องมี CPU แยกคนละตัวเสมอหรือไม่?

Lab แบ่งงานเป็น browser, API และ database

ในคาบนี้ React ทำงานใน browser; Vite เป็น development server บนเครื่องผู้พัฒนา. Express ทำงานเป็น Node.js process อีกตัว

MongoDB ทำงานใน container. ข้อมูลฐานข้อมูลเก็บใน named volume ที่เชื่อมกับ `/data/db` ภายใน container

แต่ละส่วนเริ่ม หยุด และตรวจสถานะได้แยกกัน. ภาพนี้แสดงการจัดวางของแล็บ ไม่ได้บังคับรูปแบบ production



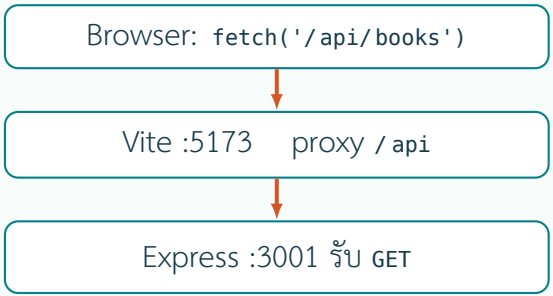
ลองคิดหรือทดลอง: React, Express และ MongoDB อยู่ใน process เดียวกันหรือไม่?

Browser ส่ง HTTP ผ่าน Vite proxy ระหว่างพัฒนา

เมื่อหน้า React เรียก `fetch('/api/books')`, browser ส่งคำขอไปยัง origin ของ Vite เช่น `localhost:5173`

ถ้า `vite.config.ts` ตั้ง proxy สำหรับ `/api`, Vite ส่งคำขอต่อไป Express ที่ `localhost:3001`. หากยังไม่ได้ตั้ง proxy ต้องตรวจ URL และ CORS ตามวิธีเชื่อมที่ใช้

เปิด DevTools → Network ดู Request URL, status และ response. Proxy นี้เป็นการตั้งค่าสำหรับ development



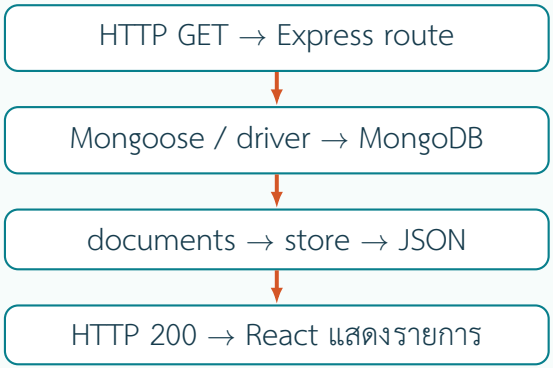
ลองคิดหรือทดลอง: ถ้า DevTools ไม่เห็นคำขอ การตรวจ database เป็นจุดเริ่มต้นที่เหมาะหรือไม่?

Express แปล HTTP เป็นการอ่าน MongoDB

Express จับคู่ `GET /api/books` กับ route, ตรวจ `limit/offset` แล้วเรียก store. Mongoose ส่งคำขอฐานข้อมูลผ่าน MongoDB driver

MongoDB อ่าน documents แล้วส่งผลกลับ. Store แปลง `_id` เป็น `id`; Express ส่ง JSON รูป `{data, pagination}` พร้อม HTTP status

MongoDB ไม่ได้ส่ง response HTTP ให้ browser โดยตรงในเส้นทางนี้. Express เป็นตัวกำหนดรูปแบบ API



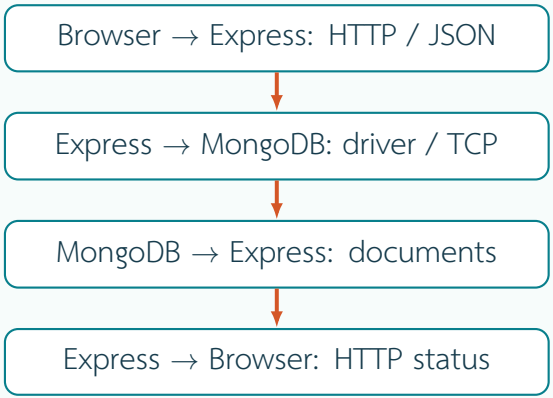
ลองคิดหรือทดลอง: ส่วนใดกำหนด JSON ที่ browser ได้รับ และส่วนใดเก็บ documents?

HTTP และ MongoDB ใช้คนละการเชื่อมต่อ

Browser ติดต่อ Express ด้วย HTTP: method, path, status และ JSON เป็นส่วนของ API ที่เราออกแบบ

Express ติดต่อ MongoDB ผ่าน MongoDB driver โดยใช้โปรโตคอลของ MongoDB บน TCP. การเชื่อมต่อนี้ใช้ URI และ port ของฐานข้อมูล ไม่ใช่ HTTP endpoint ของ browser

หาก React ได้ 500 ให้ตรวจได้ทั้งการเชื่อมต่อ HTTP และการเชื่อมต่อฐานข้อมูล โดยใช้ log และ request ID ประกอบ



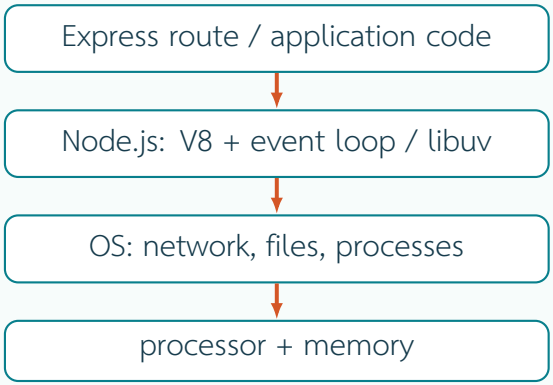
ลองคิดหรือทดลอง: ถ้า MongoDB port ไม่ตอบ คำขอ HTTP จาก browser อาจได้รับผลอย่างไร?

Node.js เป็น process ที่รัน JavaScript และจัดการ I/O

Node.js เป็น runtime; V8 ใช้รัน JavaScript. Express เป็น library ที่ทำงานภายใน Node.js process ไม่ใช่ระบบปฏิบัติการแยก

เมื่อ route รอ MongoDB ด้วย `await`, Node.js จัดการงาน I/O แบบ asynchronous. งานเครือข่ายอาศัยระบบปฏิบัติการ; งานบางชนิดใช้ worker pool

โค้ด JavaScript ที่ใช้ CPU นานบน event loop อาจทำให้คำขออื่นรอนานขึ้น แม้ API จะใช้ `async`



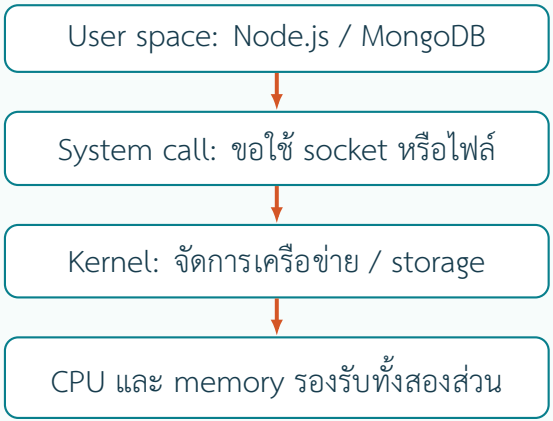
ลองคิดหรือทดลอง: เหตุใดฟังก์ชันที่คำนวณนานบน event loop จึงกระทบผู้ใช้คนอื่น?

Process ใช้ system call เพื่อขอบริการจาก kernel

Process คือโปรแกรมที่กำลังทำงาน พร้อมหน่วยความจำและทรัพยากรที่ระบบจัดให้. Express/Node.js และ MongoDB เป็นคนละ process

โค้ดผู้ใช้ทำงานใน user space. เมื่อต้องรับส่งเครือข่ายหรืออ่านไฟล์ runtime ใช้ system call เพื่อขอบริการจาก kernel

Processor รันทั้งคำสั่ง user space และ kernel; ภาพชั้นซอฟต์แวร์บอกขอบเขตหน้าที่ ไม่ได้หมายความว่าโค้ดทุกบรรทัดผ่าน system call



ลองคิดหรือทดลอง: เมื่อ Node.js ส่งข้อมูลผ่าน network ใครจัดการ socket และใครรันคำสั่ง?

ชื่อ localhost ขึ้นกับ network namespace

ใน Lab 10, Express รันบน host จึงเชื่อม `127.0.0.1:27018`; Docker ส่งต่อไป MongoDB port `27017` ใน container

ภายใน container หนึ่ง `localhost` หมายถึง network namespace ของ container นั้น ไม่ใช่ host โดยอัตโนมัติ

ถ้าภายหลังย้าย Express ไปเป็นอีก service ใน Compose network เดียวกัน ปกติจะใช้ชื่อบริการ เช่น `db:27017` ตามการตั้งค่า network



Express ใน Compose net-
work เดียวกัน: `db:27017`

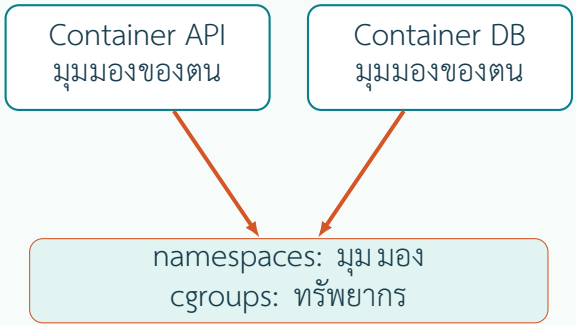
ลองคิดหรือทดลอง: ถ้า Express ย้ายเข้า container เหตุใด localhost:27018 อาจไม่ใช่ที่อยู่ MongoDB?

Namespaces และ cgroups ให้การแยกคนละด้าน

Linux namespaces จัดมุมมองของทรัพยากร เช่น process IDs, mount points และ network. Container จึงเห็นสภาพแวดล้อมตามที่กำหนด

Control groups หรือ **cgroups** ใช้จัดกลุ่มและกำหนดขีดจำกัดทรัพยากร เช่น memory หรือ CPU ตามการตั้งค่า

Containers ใช้ kernel ร่วมกัน; การแยกขึ้นกับ kernel และ configuration. อย่าถือว่า container เป็น security boundary เดียวที่เพียงพอ



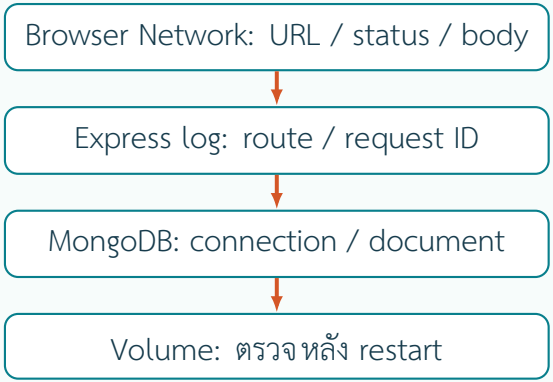
ลองคิดหรือทดลอง: การแยก network กับการจำกัด memory ใช้กลไกเดียวกันหรือไม่?

ตรวจระบบตามขอบเขตเพื่อหาสาเหตุได้เร็วขึ้น

เริ่มจากหลักฐานของคำขอเดียว: Network ใน browser → log และ request ID ของ Express → การเชื่อมต่อ MongoDB → document ในฐานข้อมูล

HTTP 422 แปลว่า API ปฏิเสธข้อมูล; 404 แปลว่าทรัพยากรที่ route ค้นไม่พบ; การเชื่อมฐานข้อมูลล้มเหลวอาจส่ง 500 ตาม error handler ของแอป

เก็บ timestamp และ status แล้วตรวจแต่ละขอบเขต. HTTP 200 จาก API เพียงอย่างเดียวไม่พิสูจน์ว่า MongoDB เก็บข้อมูลถาวร



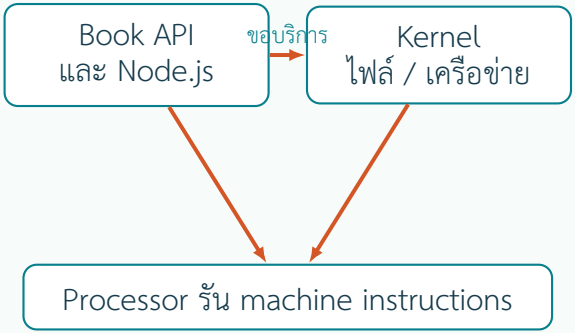
ลองคิดหรือทดลอง: ถ้า API ตอบ 422 แล้ว คำขอนั้นจำเป็นต้องถึง MongoDB หรือไม่?

Processor รันทั้งโค้ดโปรแกรมและโค้ด kernel

Node.js เป็น process ที่ใช้ V8 รัน JavaScript. V8 แปล JavaScript เป็น bytecode และอาจคอมไพล์บางส่วนเป็น machine code; processor รัน machine instructions ของ runtime และโค้ดที่คอมไพล์แล้ว

เมื่อโปรแกรมต้องการใช้บริการของระบบ เช่น เปิดไฟล์หรือรับข้อมูลเครือข่าย โปรแกรมเรียกใช้ kernel; processor จึงรันโค้ด kernel เพื่อจัดการงานนั้นด้วย

ภาพ stack ช่วยจัดหน้าที่ แต่ไม่ได้หมายความว่าโค้ดทุกบรรทัดต้องผ่าน kernel ก่อนถึง processor



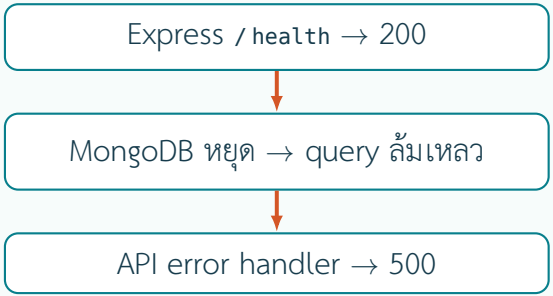
ลองคิดหรือทดลอง: เมื่อ API ขออ่านไฟล์ ส่วนใดช่วยจัดการไฟล์ และส่วนใดรันคำสั่ง?

Health endpoint และ database เป็นคนละจุดตรวจ

ใน starter ของ Lab 10, `GET /health` ตอบสถานะจาก Express. หลังเริ่มต้นแล้ว MongoDB อาจหยุดในภายหลังได้

ดังนั้น `/health` ตอบ 200 ไม่ได้พิสูจน์ว่า query MongoDB สำเร็จ. ลอง GET หนังสือและตรวจ log กับสถานะ database เพิ่ม

กำหนดความหมายของ health check ให้ชัด: liveness บอกว่า process ตอบได้; readiness อาจตรวจ dependencies ที่จำเป็นต่อการรับงาน



ลองคิดหรือทดลอง: หาก health ตอบ 200 แต่ GET books ได้ 500 ควรตรวจส่วนใดต่อ?

แนวคิดการแยกโปรแกรมเกิดก่อน Docker

ปี 2000 ระบบ FreeBSD มี **jail** เพื่อแยกพื้นที่ทำงานของผู้ใช้และบริการบนระบบเดียวกัน

ต่อมา Linux เพิ่มความสามารถแยกมุมมองและจำกัดทรัพยากร; โครงการ LXC รวมเครื่องมือสำหรับ Linux containers ราวปี 2008

ปี 2013 Docker ช่วยให้การสร้าง แจกจ่าย และเปิด container images เป็นขั้นตอนที่ทีมพัฒนานำไปใช้ร่วมกันได้ง่ายขึ้น

อ่านประวัติ Linux containers อ่านที่มา Docker



การแยกโปรแกรม → เครื่องมือสำหรับสร้างและแจกจ่าย

ลองคิดหรือทดลอง: Docker สร้างแนวคิดการแยกโปรแกรมขึ้นมาเป็นครั้งแรกหรือไม่?

เครื่องใหม่อาจมีชั้นซอฟต์แวร์ไม่เหมือนกัน

เครื่องหนึ่งมี Node.js รุ่นที่โปรเจกต์ต้องใช้ แต่อีกเครื่องอาจมีอีกรุ่นหรือยังไม่มี Node.js

ถ้าเราคัดลอกเฉพาะ `app.js` แล้วลืมแพ็กเกจ Express โปรแกรมอาจเริ่มไม่ได้ แม้ไฟล์โค้ดจะเหมือนกัน

ต้องตรวจรุ่น Node.js, package lockfile, ตัวแปรแวดล้อม และที่อยู่ database ก่อนทดสอบ API



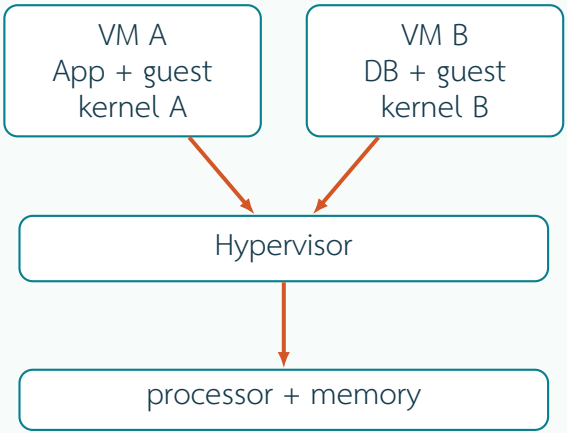
ลองคิดหรือทดลอง: นอกจากไฟล์โค้ด Book API ต้องเตรียมอะไรบนเครื่องใหม่?

เครื่องเสมือนมี kernel ของตัวเอง

Virtual machine (VM) จำลองเครื่องให้ระบบปฏิบัติการอีกชุดทำงาน. แต่ละ VM มี guest operating system และ kernel ของตน

Hypervisor จัดสรร CPU, memory และอุปกรณ์เสมือนให้แต่ละ VM. Guest OS ภายในแต่ละ VM จัดการ process ของตน

VM เหมาะกับหลายงาน โดยเฉพาะเมื่อจำเป็นต้องมีระบบปฏิบัติการหรือ kernel แยก



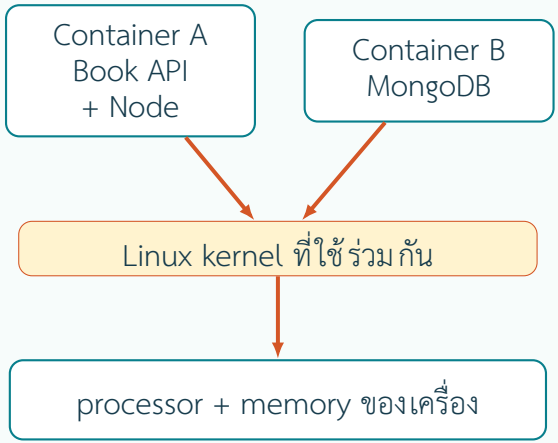
ลองคิดหรือทดลอง: ถ้า VM สองตัวใช้ระบบปฏิบัติการต่างกัน แต่ละตัวมี kernel ของใคร?

Container ใช้ kernel ร่วมแต่แยกมุมมองของโปรแกรม

Linux container เป็นกระบวนการที่รันโดยมีมุมมองไฟล์และเครือข่ายแยกตามการตั้งค่า. หลาย container บน Linux host เดียวกันใช้ kernel เดียวกัน

Image เตรียมแอป runtime และไฟล์ที่ต้องใช้; container เป็น instance ที่สร้างจาก image นั้น

บน Windows หรือ macOS Docker Desktop มักรัน Linux containers ภายใน Linux VM ที่จัดการให้



ลองคิดหรือทดลอง: ชี้ Node.js ที่อยู่กับ Book API และชี้ kernel ที่ containers ใช้ร่วมกัน

ช่วงที่ 2 / 4

Linux containers ใช้ kernel ร่วมกัน

Image บรรจุไฟล์แอปและ dependencies สำหรับสร้าง container

Browser → Express → MongoDB → Storage

Image ใช้สร้าง container ได้หลายตัว

Image ระบุไฟล์ตั้งต้นของแอปและ dependencies ซึ่งใช้สร้าง container ซ้ำได้

ในคอมพิวเตอร์ **image** เป็นชุดไฟล์ตั้งต้นสำหรับสร้าง container. อาจมี Book API, Node.js และแพ็กเกจที่แอปต้องใช้

เมื่อสร้าง container จาก image เราได้ instance ที่เปิด หยุด หรือลบได้



ลองคิดหรือทดลอง: อธิบายความต่างระหว่าง image และ container ด้วยตัวอย่าง Book API

Volume ช่วยเก็บข้อมูลเมื่อเปลี่ยน container

ข้อมูลที่เขียนใน writable layer ของ container จะหายเมื่อ container นั้นถูกลบ

Named volume เป็นพื้นที่ข้อมูลที่จัดการแยกจาก container และสามารถ mount กลับเข้า container ใหม่

ใน Lab 10 MongoDB เขียนไฟล์ฐานข้อมูลลง volume. การหยุดหรือลบ container ไม่จำเป็นต้องลบ volume นั้น



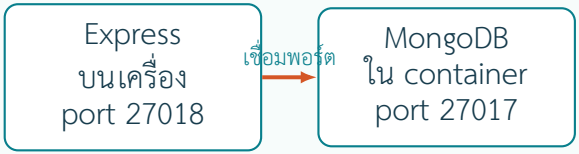
ลองคิดหรือทดลอง: ถ้าต้องการให้หนังสือยังอยู่หลังสร้าง MongoDB container ใหม่ ควรเก็บข้อมูลที่ใด?

ทางเข้าร้านคล้าย port ที่แอปใช้ติดต่อ

บริการเครือข่ายต้องฟัง port และ client ต้องเชื่อมไปยัง address กับ port ที่เข้าถึงได้

โปรแกรมใช้ port แยกช่องทางเครือข่าย. ใน Lab 10 เครื่องของเราใช้ port `27018` ไปยัง port `27017` ใน MongoDB container

ตัวเลขสองฝั่งอาจต่างกัน. เราต้องอ่านทิศทางจากเครื่องไปยัง container ให้ถูก




```
127.0.0.1:27018:27017
```

ลองคิดหรือทดลอง: ถ้า Express อยู่บนเครื่อง ต้องติดต่อ port 27018 หรือ 27017?

ช่วงที่ 3 / 4

Docker ช่วยสร้างและดูแล containers

เข้าใจเครื่องมือก่อนพิมพ์คำสั่ง

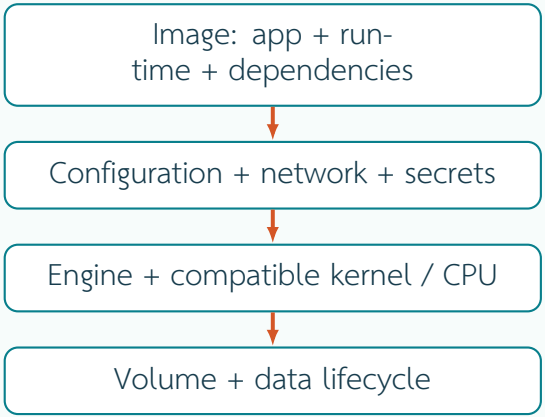
Browser → Express → MongoDB → Storage

Image ลดความต่าง แต่ยังต้องตรวจสภาพแวดล้อม

Image ระบุไฟล์และ dependencies แต่แอปยังต้องใช้ configuration, credentials, network, port และ storage จากสภาพแวดล้อมที่รัน

ตรวจความเข้ากันของ CPU architecture และ kernel กับ image; บน Docker Desktop สำหรับ macOS/Windows, Linux containers ทำงานใน Linux VM

สำหรับ Lab 10 ต้องตรวจทั้ง MongoDB container, connection URI, port และ volume. เก็บ credentials ใน environment หรือ secret manager ตามบริบท



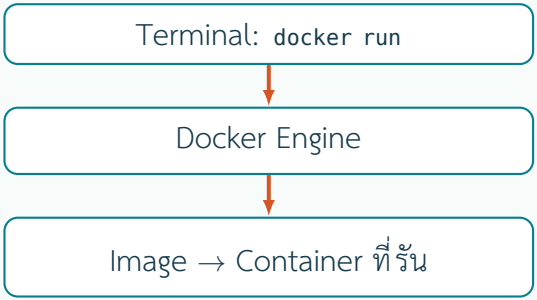
ลองคิดหรือทดลอง: image เดียวกันรับประกันว่า database URL และข้อมูลจะถูกต้องทุกเครื่องหรือไม่?

Docker รับคำสั่งเพื่อจัดการ image และ container

Docker CLI รับคำสั่งจาก terminal แล้วส่งไปยัง Docker Engine ซึ่งจัดการ images และ containers

Image อาจดาวน์โหลดจาก registry เช่น Docker Hub. Engine ใช้ image สร้าง container ตามการตั้งค่าที่เราระบุ

Docker เป็นเครื่องมือสำหรับสร้าง แจกจ่าย และจัดการ images และ containers



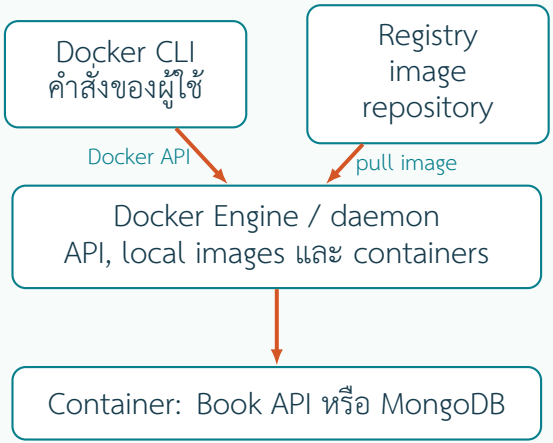
ลองคิดหรือทดลอง: ถ้า Engine ยังไม่เปิด คำสั่งจาก terminal จะสร้าง container ได้หรือไม่?

CLI ส่งคำสั่งให้ Engine และ Engine ดึง image จาก registry

Registry เป็นที่เก็บ image ให้ทีมแบ่งปัน เช่น Docker Hub. ชื่อ `mongo:7.0` ระบุ image และ tag ที่ต้องการ

`docker pull` ให้ Engine ดาวน์โหลด image ไปเก็บบนเครื่องที่รัน Engine. Docker CLI ติดต่อ Engine ในเครื่องหรือ Engine ระยะไกลที่ตั้งค่าไว้

Engine ใช้ image ที่มีอยู่สร้างและจัดการ containers; การรัน container ไม่ได้หมายความว่ามันอยู่บน registry



ลองคิดหรือทดลอง: เมื่อดาวน์โหลด image แล้ว container กำลังรันใน registry หรือบน Engine ที่เราเลือก?

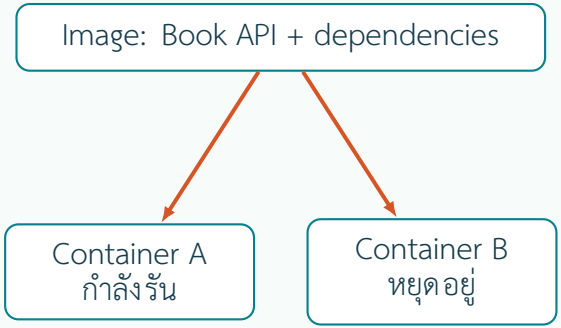
Image เป็นชุดตั้งต้น ส่วน container เป็น instance

Image เป็นชุดไฟล์และ metadata สำหรับสร้าง container.

Container เป็น instance ที่มีสถานะของตน เช่น **กำลังรัน** หรือ **หยุด**

Image เดิมใช้สร้าง container ได้หลายตัว. การแก้ไฟล์ใน container ไม่ได้แก้ image ตั้งต้นโดยอัตโนมัติ

ตรวจ image ด้วย `docker image ls`; ตรวจ containers ด้วย `docker ps -a`



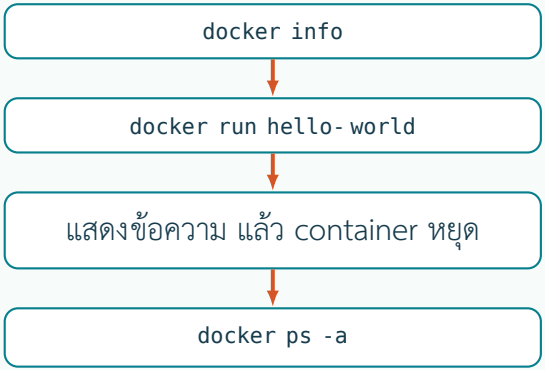
ลองคิดหรือทดลอง: `docker image ls` กับ `docker ps -a` บอกสิ่งเดียวกันหรือไม่?

คำสั่งแรกสร้าง container แล้วให้มันทำงาน

`docker run hello-world` ให้ Docker หา image, สร้าง container และรันโปรแกรมในนั้น

โปรแกรมนี้แสดงข้อความแล้วจบ. `docker ps` แสดงตัวที่กำลังรัน ส่วน `docker ps -a` แสดงตัวที่หยุดแล้วด้วย

เริ่มด้วย `docker version` และ `docker info` เพื่อตรวจว่าเครื่องติดต่อ Engine ได้



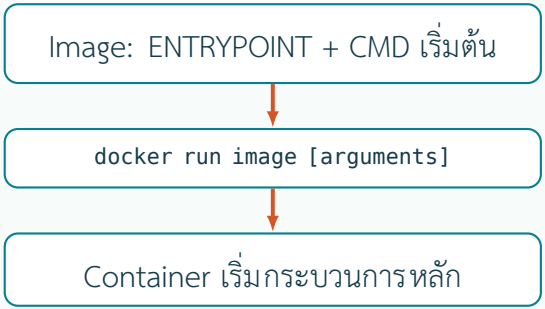
ลองคิดหรือทดลอง: ทำไม `docker ps` อาจไม่เห็น container ที่ทำงานเสร็จแล้ว?

คำสั่งแรกของ container มาจาก image หรือผู้ใช้

Image ระบุคำสั่งเริ่มต้นไว้ได้ด้วย `ENTRYPOINT` และ `CMD` เมื่อใช้ `docker run image` Docker ใช้ค่าที่ image ระบุ

ผู้ใช้เพิ่มคำสั่งหรือ arguments หลังชื่อ image ได้ หาก image มี ENTRYPOINT อยู่ ค่าเหล่านี้มักกลายเป็น arguments ของ ENTRYPOINT แทน CMD เดิม

Compose กำหนด `command:` เพื่อเปลี่ยน CMD ได้ ตรวจ image และผลจริงก่อนสรุปว่าคำสั่งใดจะรัน



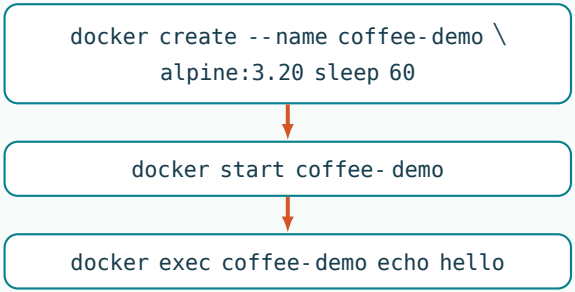
ลองคิดหรือทดลอง: ถ้า image มี ENTRYPOINT ค่า arguments หลังชื่อ image ส่งให้ส่วนใด?

คำสั่งที่ใช้จริงแสดงความต่างระหว่าง create และ exec

ตัวอย่างนี้ใช้ image `alpine:3.20` คำสั่ง `create` เตรียม container ชื่อ `coffee-demo` ให้รัน `sleep 60` แต่ยังไม่เริ่ม

`start` เริ่ม container เดิม ขณะที่มันยังรัน `exec` สั่ง `echo hello` เพิ่มภายใน container เดียวกัน

เมื่อ `sleep` จบ container จะหยุด ลบตัวทดลองด้วย `docker rm coffee-demo`



ลองคิดหรือทดลอง: หลังใช้ exec มี container ใหม่เพิ่มขึ้นหรือไม่? ตรวจด้วย `docker ps -a`

Exec เพิ่มคำสั่งใน container ที่กำลังรัน

`docker run` สร้าง container ใหม่แล้วเริ่มกระบวนการหลัก.
`docker start` เปิด container ที่สร้างไว้แล้วด้วยการตั้งค่าเดิม

`docker exec` เรียกคำสั่งเพิ่มใน container ที่กำลังรัน เช่น `docker exec -it dti201-web sh` เพื่อเข้า shell หาก image มี shell

เมื่อออกจาก shell ไม่ได้หมายความว่าเราได้สร้าง container ใหม่. ถ้า container หยุดอยู่ ต้องเริ่มมันก่อนจึงใช้ exec ได้

`run`: สร้าง + เริ่มตัวใหม่

`start`: เริ่มตัวเดิมที่หยุด

`exec`: คำสั่งเพิ่มในตัวที่รัน

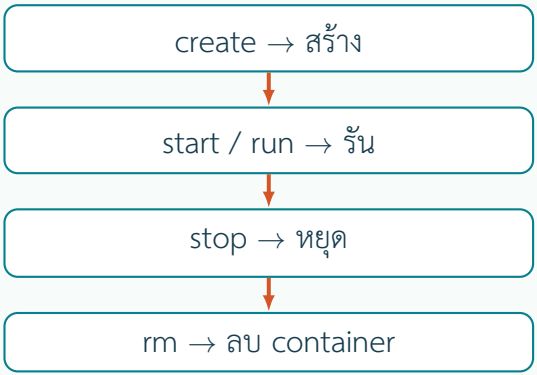
ลองคิดหรือทดลอง: ถ้าต้องการดูไฟล์ใน container ที่กำลังรัน ควรสร้างตัวใหม่หรือใช้ exec?

คำสั่ง Docker ติดตามวงจรชีวิตของ container

`docker ps -a` ดูสถานะ; `docker logs` ดูข้อความจากกระบวนการหลัก

`stop` ขอให้หยุด; `start` เปิดตัวเดิม; `rm` ลบ container ที่หยุดแล้ว. Image ตั้งต้นยังคงอยู่

`docker run --rm` ให้ Docker ลบ container นั้นหลังคำสั่งจบ; volume ที่ระบุแยกไว้ไม่ได้ถูกลบเพียงเพราะ container จบ



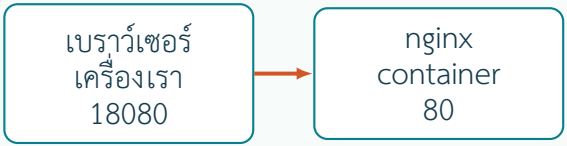
ลองคิดหรือทดลอง: container หยุดแล้ว เราใช้คำสั่งใดเปิดตัวเดิมโดยไม่สร้างตัวใหม่?

เปิดเว็บตัวอย่างก่อนกลับไป Book API

ในใบงาน เราเปิด nginx ซึ่งเป็นเว็บเซิร์ฟเวอร์ใน container. เครื่องของเราเข้า port `18080`; ภายใน container nginx ฟัง port `80`

เปิดหน้าเว็บ ตรวจด้วย `docker ps` แล้วลองหยุดและเปิด container เดิม

ขั้นนี้ทำให้เราเห็น port และวงจรชีวิต ก่อนนำแนวคิดเดียวกันไปอ่าน MongoDB ของ Lab 10




```
-p 127.0.0.1:18080:80
```

ลองคิดหรือทดลอง: ถ้าเว็บไม่ขึ้น จะตรวจ Engine, container และ port ตามลำดับอย่างไร?

ช่วงที่ 4 / 4

Compose ช่วยเขียนแผนเปิดบริการหลายส่วน

จากไฟล์บริการสู่ MongoDB และพื้นที่เก็บข้อมูล

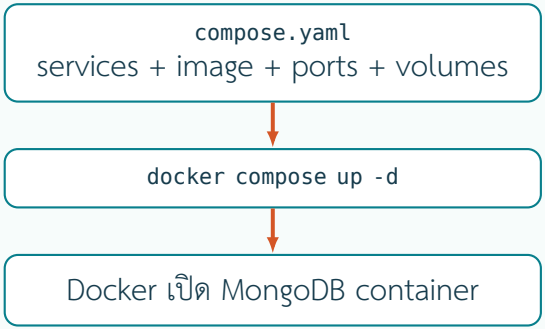
Browser → Express → MongoDB → Storage

ไฟล์ Compose ระบุสิ่งที่ต้องการให้ Docker เปิด

ไฟล์ `compose.yaml` เก็บรายละเอียดของบริการ. ใน Lab 10 บริการชื่อ `db` ใช้ image `mongo:7.0`

`ports` เชื่อม port ของเครื่องกับ port ใน container. `volumes` บอกพื้นที่ที่เก็บข้อมูล MongoDB

คำสั่ง `docker compose up -d` อ่านไฟล์แล้วสร้างหรือเปิดบริการที่กำหนด



ลองคิดหรือทดลอง: ไฟล์แผนอยู่ที่ใด และคำสั่งใดใช้แผนนั้นเปิดบริการ?

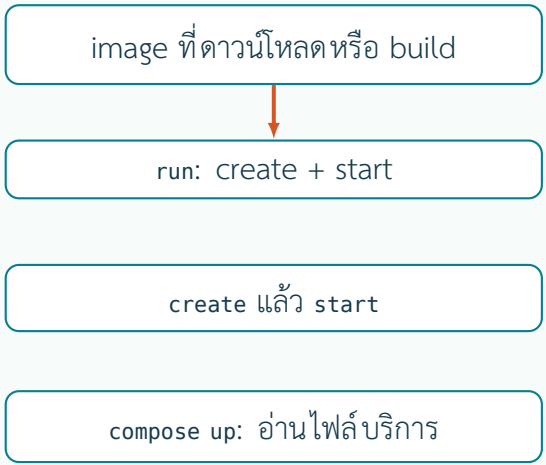
จาก image เราสร้าง container ได้หลายทาง

ทางที่ 1: `docker run` สร้าง container และเริ่มทำงานในคำสั่งเดียว

ทางที่ 2: `docker create` สร้าง container ที่ยังไม่รัน แล้ว `docker start` จึงเริ่ม container นั้น

ทางที่ 3: `docker compose up` อ่านไฟล์ Compose แล้วสร้างหรือใช้ containers ตามบริการ. ทั้งสามทางใช้ image เป็นจุดตั้งต้น

`docker build` สร้าง **image** จาก Dockerfile; ยังไม่ใช่การรัน container



ลองคิดหรือทดลอง: ถ้าใช้ `docker build` อย่างเดียว มี container กำลังรันแล้วหรือยัง?

Compose ก็เปิดคำสั่งใน container ได้สองแบบ

`docker compose run SERVICE COMMAND` สร้าง container ชั่วคราวสำหรับคำสั่งนั้นตามการตั้งค่าบริการ; เติม `--rm` เมื่อต้องการลบ container หลังจบ

`docker compose exec SERVICE COMMAND` ใช้ container ของบริการที่กำลังรันอยู่. เช่น `docker compose exec db mongosh` เปิด shell ของ MongoDB ในบริการ `db`

`docker compose up` เริ่มบริการด้วยคำสั่งที่กำหนดใน image หรือ Compose; แต่ละวิธีมีจุดประสงค์ต่างกัน

`compose up`
เปิด บริการ ตาม แผน

`compose run`
สร้าง ตัว ใหม่ สำหรับ คำ สั่ง

`compose exec`
สั่ง เพิ่ม ใน ตัว ที่ กำลัง รัน

ลองคิดหรือทดลอง: ถ้า MongoDB บริการ db กำลังทำงานและต้องการเปิด mongosh ควรใช้วิธีใด?

ไฟล์ MongoDB บอก image พอร์ตและ volume ทีละบรรทัด

ใน Lab 10 `db` เป็นชื่อบริการ. `image` ระบุ MongoDB รุ่น 7.0

`ports` เชื่อม `27018` บนเครื่องไป `27017` ใน container; `volumes` เชื่อมชื่อ `book_mongodata` ไปยัง `/data/db`

รายการ `volumes` ท้ายไฟล์ประกาศ named volume ให้ Compose จัดการ. เปิดไฟล์จริงของ Lab 10 เทียบทีละบรรทัด

```
services:
  db:
    image: mongo:7.0
    ports:
      - "127.0.0.1:27018:27017"
    volumes:
      - book_mongodata:/data/db
volumes:
  book_mongodata:
```

ลองคิดหรือทดลอง: ถ้า Express อยู่บน host ให้ใช้ port ใดติดต่อ MongoDB?

แผน Compose ไม่ได้บรรจุข้อมูลหนังสือ

Compose ระบุว่าจะเปิด MongoDB อย่างไร; ส่วนข้อมูลหนังสือ 200 เล่มมาจากไฟล์ `books.js` และ seed script ของ Lab 10

เมื่อ seed script เขียน documents ลง MongoDB ข้อมูลจึงอยู่ในฐานข้อมูลและ volume

แยกสามสิ่งนี้ให้ชัด: ไฟล์แผนบริการ, โปรแกรม seed และข้อมูลที่บันทึกแล้ว



ลองคิดหรือทดลอง: เปิด MongoDB สำเร็จแล้ว เหตุใดยังอาจไม่เห็นหนังสือ 200 เล่ม?

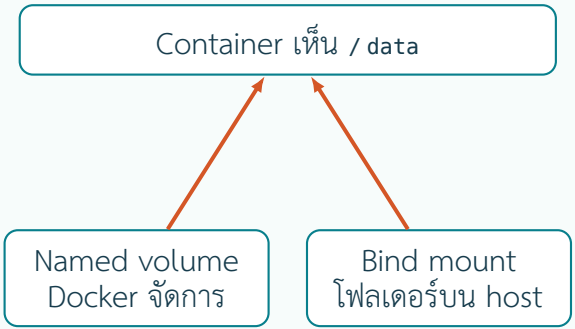
Volume และ bind mount เก็บข้อมูลไว้นอก container

ทั้งสองวิธีเชื่อมพื้นที่นอกชั้นเขียนของ container เข้ามายังตำแหน่งใน container

Named volume: Docker จัดการพื้นที่ เช่น `book_mongodata:/data/db` สำหรับไฟล์ MongoDB

Bind mount: เลือกโฟลเดอร์จริงบนเครื่องที่รัน Docker Engine เช่น `/path/books:/data`. ทั้งสองฝั่งเห็นไฟล์เดียวกัน; หาก Engine อยู่ระยะไกล โฟลเดอร์ต้องอยู่บนเครื่องนั้น

การลบ container ไม่ได้ลบโฟลเดอร์ host หรือ named volume โดยอัตโนมัติ แต่การลบไฟล์ในพื้นที่ที่ mount มีผลกับข้อมูลนั้นจริง



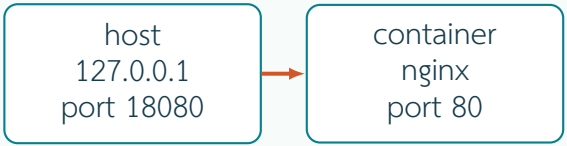
ลองคิดหรือทดลอง: ถ้าต้องแก้ source ในโฟลเดอร์เครื่องแล้วให้ container เห็นทันที วิธีใดเหมาะ?

Port mapping ระบุช่องทางจาก host ไป container

ใน `-p 127.0.0.1:18080:80` ค่าแรกเป็น address ที่รับบน host, 18080 เป็น port บน host และ 80 เป็น port ใน container

สำหรับ MongoDB ของ Lab 10 ให้ Express บน host ต่อ `127.0.0.1:27018`; Docker ส่งต่อไป port 27017 ของ MongoDB container

ถ้า host port ถูกใช้แล้ว ให้เลือก port ฝั่ง host ที่ว่าง และแก้ URL ของ client ให้ตรง



host address : host
port : container port

ลองคิดหรือทดลอง: เบราว์เซอร์บน host ต้องเปิด 18080 หรือ 80?

หยุดบริการกับลบข้อมูลเป็นคนละการกระทำ

`docker compose stop` หยุด containers ที่สร้างไว้. `docker compose start` เปิด containers เดิมอีกครั้ง

`docker compose down` หยุดและลบ containers กับ network ของ Compose; named volumes ยังอยู่โดยค่าเริ่มต้น

`docker compose down -v` ลบ named volumes ที่ประกาศในไฟล์ด้วย. ใช้เฉพาะเมื่อแน่ใจว่าจะลบข้อมูลทดลองนั้น

`stop` → หยุด

`down` → ลบ container

`down -v` → ลบ volume ด้วย

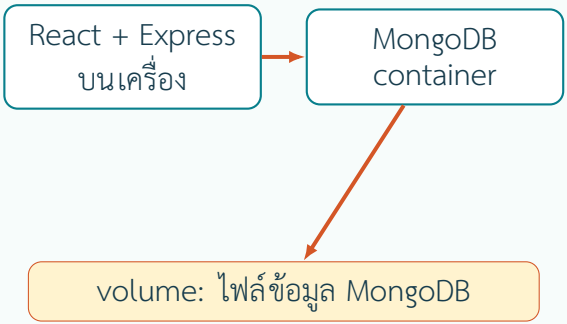
ลองคิดหรือทดลอง: ถ้าต้องเก็บข้อมูลหนังสือไว้ คำสั่งใดต้องระวังเป็นพิเศษ?

Book API กับ MongoDB ต่อกันเป็นระบบเดียว

Lab 09 สร้าง Book API ด้วย Express บน Node.js. Lab 10 เพิ่ม MongoDB เพื่อให้หนังสือยังอยู่เมื่อ Express หยุดและเปิดใหม่

ในใบงาน Docker เราเริ่มจาก nginx container และทดลอง volume ก่อน แล้วจึงอ่าน Compose ของ MongoDB

เมื่อจบ ให้ตามหนังสือเล่มเดียว: React ส่งคำขอ → Express → MongoDB → volume แล้วดูคำตอบกลับ



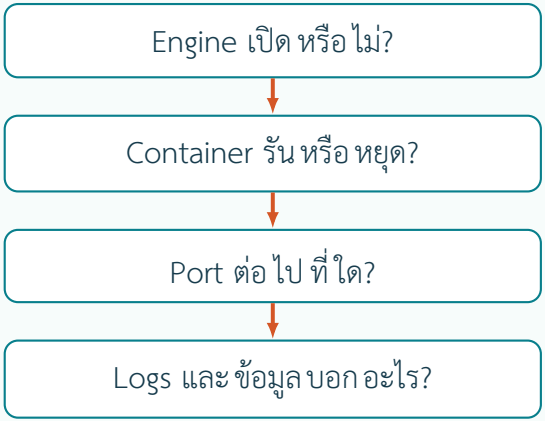
ลองคิดหรือทดลอง: ชี้ส่วนที่เป็นโปรแกรม ส่วนที่เป็น container และส่วนที่เก็บข้อมูล

เราตรวจแต่ละชั้นได้เมื่อสิ่งใดไม่ทำงาน

ถ้า Docker ติดต่อ Engine ไม่ได้ ให้ตรวจว่า Engine เปิดอยู่

ถ้า container ไม่รัน ให้ดู `docker ps -a` และ `docker logs`. ถ้าเว็บไม่ตอบ ให้ตรวจ port บนเครื่องและ port ใน container

ถ้า MongoDB เปิดแล้วแต่ยังไม่มีหนังสือ ให้ตรวจว่า seed ทำงานแล้วหรือยัง และกำลังดูฐานข้อมูลที่ถูกตัวหรือไม่



ลองคิดหรือทดลอง: ถ้า port ตอบไม่ได้ ต้องดูอะไรเป็นหลักฐานก่อนแก้?

ตรวจแต่ละชั้นด้วยหลักฐานจาก Book API

แยกหน้าที่ของ image, process, kernel, port และ volume แล้วเชื่อมกับคำขอหนังสือหนึ่งครั้ง

ในแล็บ ให้แสดงเว็บใน container, อ่านไฟล์จาก volume โดยใช้ container อีกตัว และอธิบายไฟล์ Compose ที่เขียนเอง

สุดท้ายวาด stack: Book API → Node.js → kernel → processor แล้วชี้ว่า container เพิ่มการแยกตรงไหน



ลองคิดหรือทดลอง: อธิบายด้วยคำของตนเอง: image, container และ volume ทำหน้าที่ต่างกันอย่างไร?

เอกสารอ้างอิงช่วยตรวจรายละเอียดของแต่ละชั้น

Node.js: Event Loop and asynchronous I/O

Node.js: Event Loop and worker pool

Docker Docs: Client, daemon, images and registries

Docker Docs: What is a container?

Docker Learn: containers and virtual machines

Docker Docs: Compose Quickstart

Docker Docs: Bind mounts

Docker Docs: Compose down and volumes

ใช้ใบงาน Docker แยกควบคู่กับบทเรียน แล้วกลับไป Lab 10